Project Context

This assessment was conducted as part of an internship project involving a real-world web application used within a government environment.

Executive Summary

A security assessment was conducted on a system to evaluate its resilience prior to production deployment.

The testing combined black-box and limited gray-box approaches, simulating both unauthenticated and authenticated user behavior.

Several critical vulnerabilities were identified, particularly in the file upload functionality, which allows malicious files to be uploaded and potentially executed on the server. Additional issues include DNS misconfigurations and the absence of login rate limiting.

These vulnerabilities pose significant risks such as remote code execution (RCE), service disruption, and unauthorized access. Immediate remediation is required before the system can be considered secure for production use.

Tools & Testing Approach

The security assessment was conducted using a combination of black-box and limited gray-box testing to simulate both unauthenticated and authenticated user behavior.

Testing focused on identifying common web vulnerabilities through input validation, file upload mechanisms, authentication flows, and infrastructure misconfigurations. Both manual testing and automated scanning were performed, with all findings verified to reduce false positives.

The following tools were used:

- Burp Suite (Community Edition) – HTTP request/response analysis and manual testing
- Nessus Vulnerability Scanner – Infrastructure vulnerability scanning
- Google Chrome DevTools – Network traffic inspection
- OpenSSL CLI – SSL/TLS validation
- Kali Linux – Testing environment

All findings were manually validated to ensure accuracy and reduce false positives.

Security Findings

1. Unrestricted File Upload (Critical)

Description:
The application fails to properly validate uploaded files. Multiple upload features accept dangerous file types (e.g., PHP, JSON) without verifying MIME type or content. Additionally, malicious scripts can bypass restrictions by disguising themselves as image files (e.g., .php renamed to .jpg).

Impact:
- Potential Remote Code Execution (RCE)
- Full server compromise if uploaded files are executed
- Risk of malicious file hosting and further exploitation

Evidence:
- Malicious files (PHP/JSON) were successfully uploaded in data import functionality
- A malicious file (shell.php) successfully uploaded as shell.jpg
- No MIME type validation or image verification observed

Recommendation:
- Enforce strict MIME type validation on the server side
- Implement file content verification (e.g., image validation)
- Use whitelisting for allowed file types (e.g., CSV, XLSX, JPG, PNG)
- Rename uploaded files and store them in a non-executable directory

2. DNS Amplification Vulnerability (High)

Description:
The DNS server allows responses significantly larger than the corresponding requests, making it susceptible to amplification attacks.

Impact:
- Can be abused in DDoS amplification attacks
- May lead to IP reputation damage or blacklisting
- Service disruption risks

Evidence:
- Vulnerability identified via Nessus scan
- DNS server responds to small queries with large responses

Recommendation:
- Disable open DNS recursion
- Restrict queries to trusted sources only
- Implement rate limiting on DNS requests

3. DNS Cache Poisoning Vulnerability (Medium)

Description:
The DNS server is vulnerable to cache poisoning, allowing attackers to inject malicious DNS records into the cache.

Impact:
- Redirection of users to malicious websites
- Potential phishing and Man-in-the-Middle (MITM) attacks
- Abuse of trusted domain for further attacks

Evidence:
- Weak DNS configuration identified through Nessus scanning

Recommendation:
- Disable recursion on public-facing DNS servers
- Implement DNSSEC
- Enable source port randomization
- Keep DNS software up to date

4. No Rate Limiting on Login (Medium)

Description:
The login functionality does not restrict repeated authentication attempts.

Impact:
- Vulnerable to brute-force attacks
- Increased risk of unauthorized account access

Evidence:
- Unlimited login attempts observed
- No CAPTCHA, delay, or account lockout mechanism

Recommendation:
- Implement rate limiting on login attempts
- Add account lockout after multiple failures
- Introduce CAPTCHA or progressive delays

Conclusion

The assessment revealed multiple critical vulnerabilities that significantly impact the system's security posture. The most severe issue is unrestricted file upload, which introduces a high risk of remote code execution (RCE). Addressing these vulnerabilities is essential to ensure the system is secure and ready for production deployment.